

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI

Semester: 5

Academic Year: 2026-27

Division: E

Batch: 1

Practical No: 3

Roll no: 52

PRN No: 12410371

Name of Student: Vaidik Samarth

Subject Name: Deep Learning

Problem Statement: Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

Code and Output:

```
10 | import tensorflow as tf
    | import numpy as np
    | import matplotlib.pyplot as plt
    |
    | from tensorflow.keras.models import Sequential
    | from tensorflow.keras.layers import Dense, Flatten
    | from tensorflow.keras.optimizers import Adam
    |
    | print("TensorFlow Version:", tf.__version__)
    |
    | TensorFlow Version: 2.20.0
    |
11 | # Load MNIST dataset
    | (x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
    |
    | print("Training Images:", x_train.shape)
    | print("Training Labels:", y_train.shape)
    |
    | print("Testing Images:", x_test.shape)
    | print("Testing Labels:", y_test.shape)
    |
    | Training Images: (60000, 28, 28)
    | Training Labels: (60000,)
    | Testing Images: (10000, 28, 28)
    | Testing Labels: (10000,)
```



```
[6] x_train = x_train / 255.0
x_test = x_test / 255.0

print("Normalization Complete")
```

Normalization Complete

```
[7] def create_model(learning_rate):

    model = Sequential([
        Flatten(input_shape=(28,28)),

        Dense(128, activation='relu'),

        Dense(64, activation='relu'),

        Dense(10, activation='softmax')
    ])

    optimizer = Adam(learning_rate=learning_rate)

    model.compile(
        optimizer=optimizer,
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    return model
```

[8]



```
learning_rates = [0.1, 0.01, 0.001]
epochs_list = [5,10,20]

results = []

for lr in learning_rates:
    for ep in epochs_list:
        print("="*50)
        print("Learning Rate:", lr)
        print("Epochs:", ep)

        model = create_model(lr)

        history = model.fit(
            x_train,
            y_train,
            epochs=ep,
            validation_data=(x_test,y_test),
            verbose=1
        )

        loss, accuracy = model.evaluate(x_test,y_test,verbose=0)

        print("Accuracy:", accuracy)

        results.append([lr, ep, accuracy, loss])
```

[9]
✓ Os

```
print("Final Results\n")

print("Learning Rate\tEpochs\tAccuracy\tLoss")

for r in results:

    print(f"{r[0]}\t\t{r[1]}\t\t{r[2]:.4f}\t\t{r[3]:.4f}")
```

▼

Final Results

Learning Rate	Epochs	Accuracy	Loss
0.1	5	0.1976	2.0116
0.1	10	0.1834	1.9074
0.1	20	0.1765	2.0250
0.01	5	0.9652	0.1498
0.01	10	0.9648	0.1785
0.01	20	0.9657	0.2198
0.001	5	0.9718	0.0951
0.001	10	0.9793	0.0819
0.001	20	0.9765	0.1275



